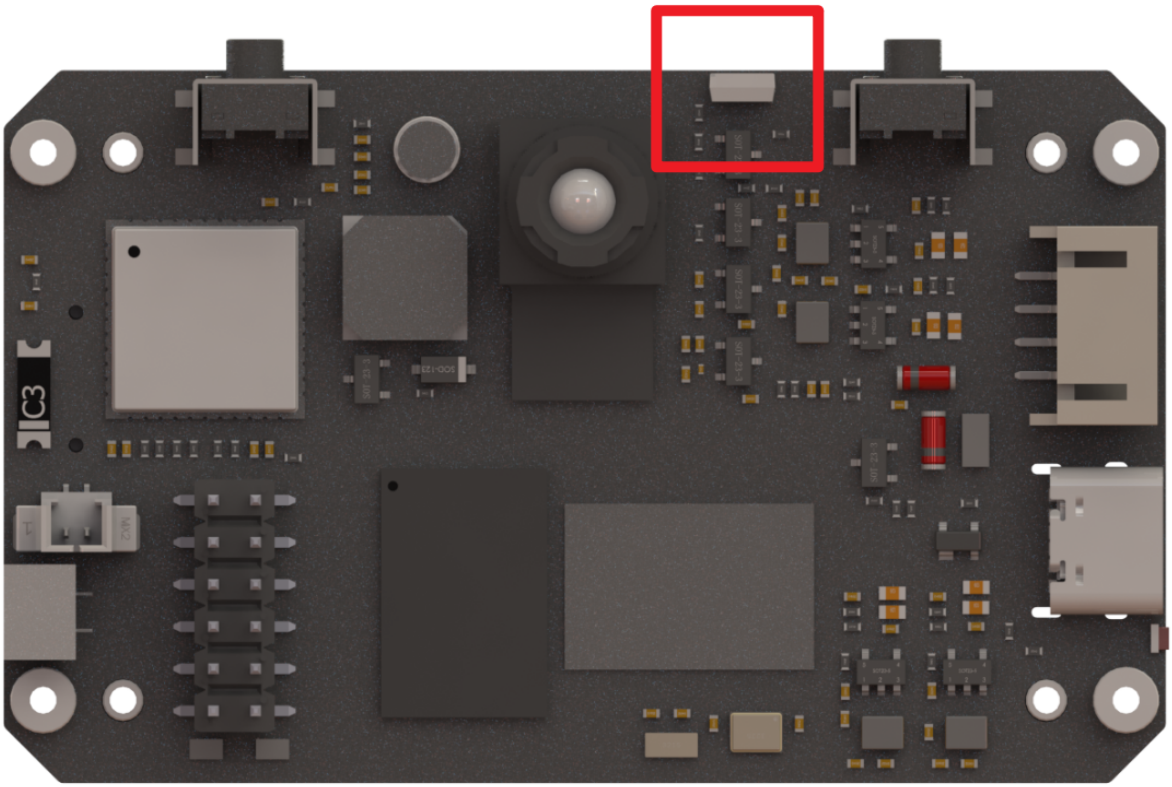
RGB 灯

RGB 灯
- 简介
- 快速开始
- 模拟呼吸灯
- WS2812 驱动
- 补充知识

简介

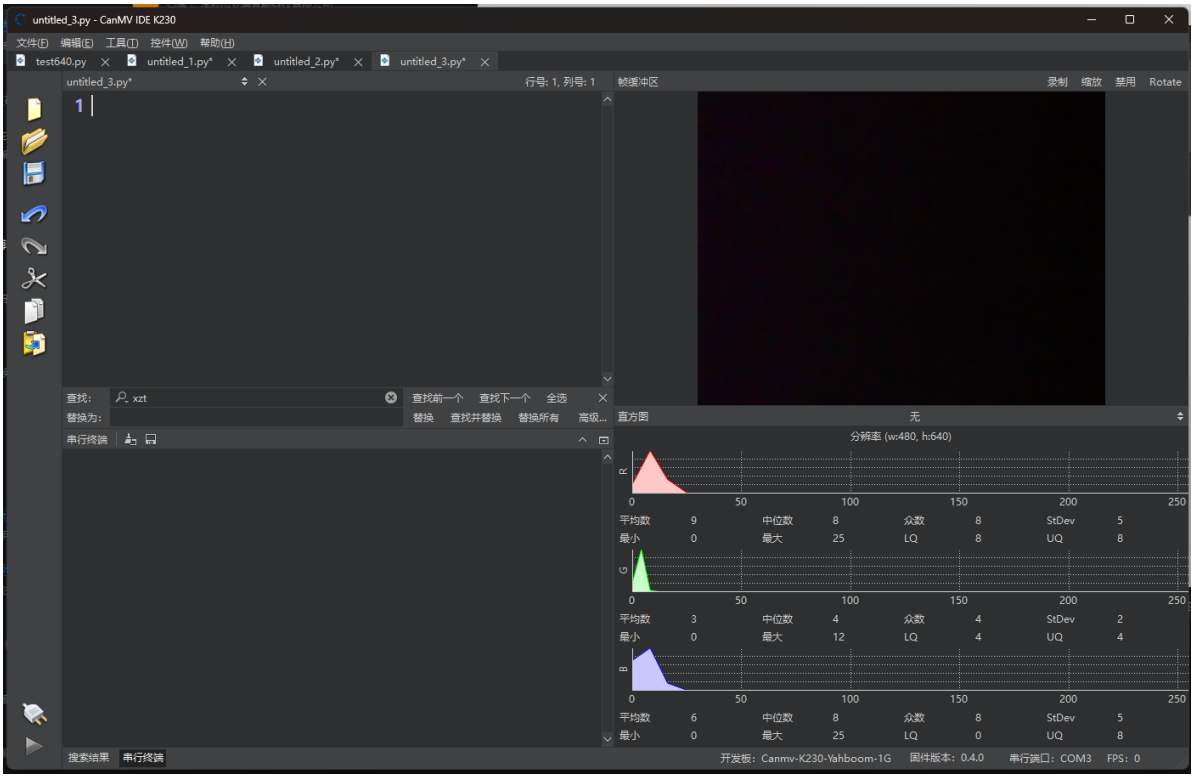
本节我们来学习如何使用代码控制 YAHBOOM K230上面的 RGB 灯



快速开始

我们打开 CanMV IDE，将 K230 连接至电脑。

按 Ctrl + N 键新建一份代码，并删除所有自动生成的代码内容



复制下面的代码并粘贴到IDE中 [源码汇总/02.Basic/02.1_rgb.py]

```
# 导入YbRGB库 (Import YbRGB library)
from ybUtils.YbRGB import YbRGB
# 导入时间库 (Import time library)
import time

# 实例化YbRGB对象 (Initialize YbRGB object)
YbRGB = YbRGB()

# 让RGB灯显示蓝色 (82, 139, 255) (Make the RGB light display blue color (82, 139, 255))
YbRGB.show_rgb((82, 139, 255))

# 程序阻塞等待3秒 (Block the program for 3 seconds)
time.sleep(3)

# 关闭RGB灯 (0,0,0) (Turn off the RGB light by setting color to (0,0,0))
YbRGB.show_rgb((0,0,0))
```

模拟呼吸灯

我们还可以结合time模块以及math模块，模拟实现一个更好看的呼吸灯效果

代码如下 [源码汇总/02.Basic/02.2_better_rgb.py]

```
# 导入YbRGB库 (Import YbRGB library)
from ybUtils.YbRGB import YbRGB
# 导入时间库 (Import time library)
import time
# 导入数学库 (Import math library)
import math

# 初始化RGB LED (Initialize RGB LED)
rgb = YbRGB()
```

```
def breath_effect(r, g, b, duration=2):
    """
    实现呼吸灯效果 (Implement breathing light effect)
    r, g, b: 目标颜色的RGB值（0-255） (Target color RGB values (0-255))
    duration: 一次呼吸效果的持续时间（秒） (Duration of one breathing cycle in
seconds)
    """
    steps = 1000  # 渐变步数 (Number of gradient steps)
    for i in range(steps):
        # 使用正弦函数使亮度变化更加平滑 (Use sine function to make brightness change
smoother)
        brightness = math.sin(i / steps * math.pi)
        current_r = int(r * brightness)
        current_g = int(g * brightness)
        current_b = int(b * brightness)
        rgb.show_rgb([current_r, current_g, current_b])
        time.sleep(duration / (2 * steps))

    # 渐暗过程 (Fade-out process)
    for i in range(steps-1, -1, -1):
        brightness = math.sin(i / steps * math.pi)
        current_r = int(r * brightness)
        current_g = int(g * brightness)
        current_b = int(b * brightness)
        rgb.show_rgb([current_r, current_g, current_b])
        time.sleep(duration / (2 * steps))

# 定义几种好看的颜色 (Define several beautiful colors)
colors = [
    (255, 0, 0),    # 红色 (Red)
    (0, 255, 0),    # 绿色 (Green)
    (0, 0, 255),    # 蓝色 (Blue)
    (255, 0, 255),  # 紫色 (Purple)
    (255, 165, 0),  # 橙色 (Orange)
    (0, 255, 255),  # 青色 (Cyan)
]

# 主循环 (Main loop)
try:
    while True:
        for color in colors:
            breath_effect(color[0], color[1], color[2])

except Exception:
    # 确保程序结束时关闭LED (Ensure LED is turned off when the program ends)
    rgb.show_rgb([0, 0, 0])
```

WS2812 驱动

> 这部分内容较为复杂，需要一定的基础知识。跳过这部分不影响后续对K230的使用

1. 基本硬件设置：

- YAHBOOM K230 使用的是WS2812B类型的RGB LED
- 我们通过 SPI（串行外设接口）与LED通信，设置波特率为2.5MHz

- LED的控制信号连接到开发板的QSPI0_D0引脚（引脚16）

2. 颜色设置原理：

```
def set_led(self, index, r, g, b):
    if 0 <= index < self._num_leds:
        self._buf[index*3] = g    # 注意顺序是GRB而不是RGB
        self._buf[index*3 + 1] = r
        self._buf[index*3 + 2] = b
```

- 每个LED需要3个字节数据，分别代表绿(G)、红(R)、蓝(B)
- 数值范围是0-255，值越大颜色越亮

3. 数据传输原理（**show**函数）：
   WS2812B LED的通信协议比较特殊：

- 每个比特"1"用高电平110和低电平000表示
- 每个比特"0"用高电平100和低电平000表示
- 因此代码中：

```
# 表示二进制1
timing_buf[pos] = 0b11100000      # 110xxxxx

# 表示二进制0
timing_buf[pos] = 0b10000000      # 100xxxxx
```

4. 数据发送过程：

- 将每个颜色字节拆分成8个比特
- 每个比特转换成相应的时序信号
- 分批发送数据（每批最大1024字节）避免内存溢出
- 发送完成后等待300微秒作为复位信号

举个例子：
如果要设置一个LED为红色（R=255, G=0, B=0）：

```
rgb = YbRGB(1)  # 创建一个LED对象
rgb.set_led(0, 255, 0, 0)  # 设置为红色
rgb.show()  # 发送数据
```

内部过程：

1. 在`set_led`中，数据被存储为：[0, 255, 0]
2. `show()`函数将这些数据转换为时序信号
3. 通过SPI接口发送给LED
4. LED接收到信号后显示相应的颜色

因为WS2812B LED使用特殊的单线通信协议来接收数据，YbRGB中的`show()`函数实际上就是在将普通的RGB值转换成LED能够理解的时序信号。

补充知识

RGB灯的RGB和电脑屏幕上的RGB显示的不一样？

这是因为 WS2812B RGB LED 和电脑显示器在显示原理上有根本的区别：

显示器的工作原理 ：

- 显示器是发光二极管或液晶面板通过阻挡或透过背光来显示颜色
- RGB(0,0,0) 表示所有颜色通道都不发光/完全阻挡背光，因此呈现黑色
- RGB(255,255,255) 表示所有颜色通道全亮/完全透过背光，呈现白色

WS2812B LED的工作原理 ：

- RGB LED 是通过向LED发送电信号来控制发光
- RGB(0,0,0) 表示不发送任何信号，LED 处于"关闭"状态
- RGB(255,255,255) 表示三个颜色通道都全功率发光，呈现白色